Angular Note 4.3: Attribute Directives

> **Module:** Directives - Structural and Attribute
> **Topic:** Working with Attribute Directives
> **Duration:** 3-4 hours

Introduction

Attribute directives modify the appearance or behavior of existing elements without changing the DOM structure. Unlike structural directives, they don't add or remove elements—they enhance what's already there. This note covers built-in attribute directives (ngClass, ngStyle) and advanced techniques using HostListener, HostBinding, and Renderer2.

---

1. ngClass - Dynamic Class Binding

1.1 Understanding ngClass

The `ngClass` directive dynamically adds or removes CSS classes based on conditions. It provides more flexibility than the simple class binding syntax.

1.2 ngClass with Object

The most common and powerful way to use ngClass is with an object where keys are class names and values are boolean expressions.

Component TypeScript (app.component.ts)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
```

```
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  isActive = true;
  hasError = false;
  isSpecial = true;
  isDisabled = false;

  toggleActive() {
    this.isActive = !this.isActive;
  }

  toggleError() {
    this.hasError = !this.hasError;
  }

  toggleSpecial() {
    this.isSpecial = !this.isSpecial;
  }
}
```

Component CSS (app.component.css)

```
.active {
  background-color: #4CAF50;
  color: white;
  font-weight: bold;
}

.error {
  border: 2px solid #f44336;
  background-color: #ffebee;
  color: #c62828;
}

.special {
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);
  border-radius: 8px;
}

.disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

```
.box {
  padding: 20px;
  margin: 10px 0;
  transition: all 0.3s ease;
}
```

Component Template (app.component.html)

```
<h3>ngClass with Object Syntax</h3>

<div class="box"
     [ngClass]="{
       'active': isActive,
       'error': hasError,
       'special': isSpecial,
       'disabled': isDisabled
     }">
  <h4>Dynamic Classes Demo</h4>
  <p>This box has multiple conditional classes</p>
  <p>
    Active: {{ isActive }} |
    Error: {{ hasError }} |
    Special: {{ isSpecial }} |
    Disabled: {{ isDisabled }}
  </p>
</div>

<div style="margin: 15px 0;">
  <button (click)="toggleActive()">Toggle Active</button>
  <button (click)="toggleError()">Toggle Error</button>
  <button (click)="toggleSpecial()">Toggle Special</button>
</div>
```

Output:

Browser Display:

Initially: A green box with white bold text and shadow effect appears (active + special classes applied)

Shows: "Active: true | Error: false | Special: true | Disabled: false"

After clicking "Toggle Active": Green background and bold text disappear, box becomes plain

After clicking "Toggle Error": Red border and light red background appear

After clicking "Toggle Special": Shadow effect disappears

All transitions are smooth due to CSS transition property.

1.3 ngClass with Array

You can pass an array of class names to apply multiple classes at once.

Component TypeScript

```
export class AppComponent {
  currentClasses = ['active', 'special'];

  applyTheme(theme: string) {
    if (theme === 'success') {
      this.currentClasses = ['active', 'special'];
    } else if (theme === 'danger') {
      this.currentClasses = ['error', 'special'];
    } else if (theme === 'warning') {
      this.currentClasses = ['warning', 'special'];
    }
  }
}
```

Component CSS

```
.warning {
  background-color: #ff9800;
  color: white;
  border: 2px solid #f57c00;
}
```

Component Template

```
<h3>ngClass with Array</h3>

<div class="box" [ngClass]="currentClasses">
  <h4>Theme Box</h4>
  <p>Current classes: {{ currentClasses.join(', ') }}</p>
</div>

<button (click)="applyTheme('success')">Success Theme</button>
<button (click)="applyTheme('danger')">Danger Theme</button>
<button (click)="applyTheme('warning')">Warning Theme</button>
```

Output:

Browser Display:

Initially: Green box with shadow (success theme)

"Current classes: active, special"

After clicking "Danger Theme": Red bordered box with light red background

"Current classes: error, special"

After clicking "Warning Theme": Orange box with orange border

"Current classes: warning, special"

1.4 ngClass with String

For simple cases, you can pass a space-separated string of class names.

```
<div class="box" [ngClass]="'active special'">
  <p>This has active and special classes</p>
</div>

<!-- Dynamic string -->
```

```
<div class="box" [ngClass]="getClassString()">
  <p>Dynamic class string</p>
</div>
```

Component TypeScript

```
getClassString(): string {
  return this.isActive ? 'active special' : 'disabled';
}
```

1.5 Practical Example: Status Cards

Component TypeScript

```
export class AppComponent {
  tasks = [
    { id: 1, title: 'Design Homepage', status: 'completed', priority: 'high' },
    { id: 2, title: 'Fix Login Bug', status: 'in-progress', priority: 'critical' },
    { id: 3, title: 'Update Documentation', status: 'pending', priority: 'low' },
    { id: 4, title: 'Code Review', status: 'in-progress', priority: 'medium' }
  ];

  getTaskClasses(task: any) {
    return {
      'status-completed': task.status === 'completed',
      'status-in-progress': task.status === 'in-progress',
      'status-pending': task.status === 'pending',
      'priority-critical': task.priority === 'critical',
      'priority-high': task.priority === 'high',
      'priority-medium': task.priority === 'medium',
      'priority-low': task.priority === 'low'
    };
  }
}
```

Component CSS

```
.task-card {
  padding: 15px;
  margin: 10px 0;
```

```
  border-radius: 8px;
  border-left: 5px solid #ccc;
  background-color: #f9f9f9;
  transition: all 0.3s;
}

.status-completed {
  background-color: #e8f5e9;
  border-left-color: #4caf50;
}

.status-in-progress {
  background-color: #e3f2fd;
  border-left-color: #2196f3;
}

.status-pending {
  background-color: #fff3e0;
  border-left-color: #ff9800;
}

.priority-critical {
  font-weight: bold;
  box-shadow: 0 0 10px rgba(244, 67, 54, 0.5);
}

.priority-high {
  border-width: 5px;
}

.priority-low {
  opacity: 0.8;
}
```

Component Template

```
<h3>Task Management Board</h3>

<div *ngFor="let task of tasks"
     class="task-card"
     [ngClass]="getTaskClasses(task)">
  <h4>{{ task.title }}</h4>
  <p>
    Status: <strong>{{ task.status }}</strong> |
```

```
    Priority: <strong>{{ task.priority }}</strong>
  </p>
</div>
```

> **Output:**
>
> **Browser Display:**
>
> 1. **Design Homepage** - Green background (completed), thick border (high priority)
> 2. **Fix Login Bug** - Blue background (in-progress), red glow shadow (critical priority), bold text
> 3. **Update Documentation** - Orange background (pending), slightly faded (low priority)
> 4. **Code Review** - Blue background (in-progress), normal appearance (medium priority)

---

2. ngStyle - Dynamic Inline Styles

2.1 Understanding ngStyle

The `ngStyle` directive allows you to set multiple inline styles dynamically using an object.

2.2 Basic ngStyle Usage

Component TypeScript

```
export class AppComponent {
  fontSize = 16;
  textColor = '#333';
  bgColor = '#f0f0f0';
  borderRadius = 5;
  padding = 20;

  increaseFont() {
    this.fontSize += 2;
  }

  decreaseFont() {
```

```
    if (this.fontSize > 10) {
      this.fontSize -= 2;
    }
  }

  changeColor(color: string) {
    this.textColor = color;
  }

  changeBg(color: string) {
    this.bgColor = color;
  }
}
```

Component Template

```
<h3>ngStyle Dynamic Styling</h3>

<div [ngStyle]="{
       'font-size.px': fontSize,
       'color': textColor,
       'background-color': bgColor,
       'border-radius.px': borderRadius,
       'padding.px': padding,
       'border': '2px solid ' + textColor,
       'transition': 'all 0.3s ease'
     }">
  <h4>Dynamically Styled Content</h4>
  <p>Font Size: {{ fontSize }}px</p>
  <p>Text Color: {{ textColor }}</p>
  <p>Background: {{ bgColor }}</p>
</div>

<div style="margin: 15px 0;">
  <button (click)="increaseFont()">Increase Font</button>
  <button (click)="decreaseFont()">Decrease Font</button>

  <br><br>

  <button (click)="changeColor('#e91e63')">Pink Text</button>
  <button (click)="changeColor('#2196f3')">Blue Text</button>
  <button (click)="changeColor('#4caf50')">Green Text</button>

  <br><br>
```

```
  <button (click)="changeBg('#fff3e0')">Orange BG</button>
  <button (click)="changeBg('#e8f5e9')">Green BG</button>
  <button (click)="changeBg('#e3f2fd')">Blue BG</button>
</div>
```

Output:

Browser Display:

A gray box appears with dark text at 16px font size.

After clicking "Increase Font" twice: Text grows to 20px

After clicking "Pink Text": Text and border turn pink

After clicking "Green BG": Background becomes light green

All changes animate smoothly due to CSS transition.

2.3 Computed Styles with Methods

Component TypeScript

```
export class AppComponent {
  temperature = 25; // Celsius

  getTemperatureStyles() {
    let bgColor: string;
    let textColor: string;
    let borderColor: string;

    if (this.temperature < 10) {
      bgColor = '#e3f2fd';
      textColor = '#0d47a1';
      borderColor = '#2196f3';
    } else if (this.temperature < 25) {
      bgColor = '#e8f5e9';
      textColor = '#1b5e20';
      borderColor = '#4caf50';
    } else if (this.temperature < 35) {
```

```
      bgColor = '#fff3e0';
      textColor = '#e65100';
      borderColor = '#ff9800';
    } else {
      bgColor = '#ffebee';
      textColor = '#b71c1c';
      borderColor = '#f44336';
    }

    return {
      'background-color': bgColor,
      'color': textColor,
      'border': `4px solid ${borderColor}`,
      'padding': '20px',
      'border-radius': '10px',
      'text-align': 'center',
      'font-size': '24px',
      'font-weight': 'bold',
      'transition': 'all 0.5s ease'
    };
  }

  adjustTemperature(change: number) {
    this.temperature += change;
  }
}
```

Component Template

```
<h3>Temperature Monitor</h3>

<div [ngStyle]="getTemperatureStyles()">
  {{ temperature }}°C
</div>

<div style="margin: 15px 0;">
  <button (click)="adjustTemperature(-5)">❄️ -5°C</button>
  <button (click)="adjustTemperature(-1)">-1°C</button>
  <button (click)="adjustTemperature(1)">+1°C</button>
  <button (click)="adjustTemperature(5)">🔥 +5°C</button>
</div>

<p style="text-align: center; color: #666;">
  <em>
    Status:
```

```
    {{ temperature < 10 ? 'Freezing ❄️' :
       temperature < 25 ? 'Cool 😊' :
       temperature < 35 ? 'Warm ☀️' : 'Hot 🔥' }}
  </em>
</p>
```

Output:

Browser Display:

A green box with "25°C" in large, bold text

Status: "Cool 😊"

After clicking "+5°C" twice (35°C):

Box turns red with dark red text and red border

Status: "Hot 🔥"

After clicking "-5°C" six times (5°C):

Box turns blue with blue text and border

Status: "Freezing ❄️"

2.4 Combining ngClass and ngStyle

You can use both directives together on the same element for maximum flexibility.

Component TypeScript

```
export class AppComponent {
  notification = {
    type: 'info',
    message: 'System update available',
    priority: 'medium',
    timestamp: new Date()
  };
```

```
  getNotificationStyles() {
    const baseStyles = {
      'padding': '15px',
      'margin': '10px 0',
      'border-radius': '8px',
      'animation': 'slideIn 0.3s ease'
    };

    if (this.notification.priority === 'high') {
      return {
        ...baseStyles,
        'border-left-width': '8px'
      };
    }

    return baseStyles;
  }

  getNotificationClasses() {
    return {
      'notification': true,
      'notification-info': this.notification.type === 'info',
      'notification-success': this.notification.type === 'success',
      'notification-warning': this.notification.type === 'warning',
      'notification-error': this.notification.type === 'error',
      'priority-high': this.notification.priority === 'high'
    };
  }

  showNotification(type: string, message: string, priority: string) {
    this.notification = {
      type,
      message,
      priority,
      timestamp: new Date()
    };
  }
}
```

Component CSS

```
.notification {
  border-left: 4px solid #ccc;
  font-size: 14px;
```

```
}

.notification-info {
  background-color: #e3f2fd;
  border-left-color: #2196f3;
  color: #0d47a1;
}

.notification-success {
  background-color: #e8f5e9;
  border-left-color: #4caf50;
  color: #1b5e20;
}

.notification-warning {
  background-color: #fff3e0;
  border-left-color: #ff9800;
  color: #e65100;
}

.notification-error {
  background-color: #ffebee;
  border-left-color: #f44336;
  color: #b71c1c;
}

.priority-high {
  font-weight: bold;
}

@keyframes slideIn {
  from {
    transform: translateX(-100%);
    opacity: 0;
  }
  to {
    transform: translateX(0);
    opacity: 1;
  }
}
```

Component Template

```
<h3>Notification System</h3>

<div [ngClass]="getNotificationClasses()"
     [ngStyle]="getNotificationStyles()">
  <strong>{{ notification.type.toUpperCase() }}:</strong> {{ notification.message }}
  <br>
  <small>{{ notification.timestamp | date:'short' }}</small>
</div>

<div style="margin: 15px 0;">
  <button (click)="showNotification('info', 'System updated successfully', 'low')">
    Info
  </button>
  <button (click)="showNotification('success', 'File uploaded', 'medium')">
    Success
  </button>
  <button (click)="showNotification('warning', 'Storage almost full', 'high')">
    Warning
  </button>
  <button (click)="showNotification('error', 'Connection failed', 'high')">
    Error
  </button>
</div>
```

Output:

Browser Display:

Blue notification box: "INFO: System update available" with timestamp

After clicking "Success":

Green notification slides in: "SUCCESS: File uploaded"

After clicking "Warning":

Orange notification with thick border (high priority): "WARNING: Storage almost full" in bold

After clicking "Error":

> Red notification with thick border: "ERROR: Connection failed" in bold

---

3. HostListener - Listening to Host Events

3.1 Introduction to HostListener

The `@HostListener` decorator allows a directive to listen to events on its host element (the element the directive is applied to).

> **Note:** This section introduces concepts that will be fully implemented when creating custom directives in the next note. We're learning the tools used to build directives.

3.2 Basic HostListener Example

```
import { Directive, HostListener, ElementRef } from '@angular/core';

@Directive({
  selector: '[appHoverHighlight]'
})
export class HoverHighlightDirective {

  constructor(private el: ElementRef) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.highlight('yellow');
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.highlight('');
  }

  private highlight(color: string) {
    this.el.nativeElement.style.backgroundColor = color;
  }
}
```

Usage in Template

```
<p appHoverHighlight>Hover over this text to see highlighting</p>
<div appHoverHighlight style="padding: 20px; border: 1px solid #ccc;">
  Hover over this box!
</div>
```

3.3 HostListener with Event Parameters

```
@Directive({
  selector: '[appClickTracker]'
})
export class ClickTrackerDirective {

  clickCount = 0;

  @HostListener('click', ['$event'])
  onClick(event: MouseEvent) {
    this.clickCount++;
    console.log(`Clicked ${this.clickCount} times`);
    console.log('Mouse position:', event.clientX, event.clientY);
  }

  @HostListener('dblclick')
  onDoubleClick() {
    console.log('Double clicked!');
    this.clickCount = 0;
  }
}
```

3.4 Common HostListener Events

Event	Triggered When	Use Case
`click`	Element is clicked	Track interactions, toggle states
`mouseenter`	Mouse enters element	Hover effects, tooltips
`mouseleave`	Mouse leaves element	Remove hover effects

Event	Triggered When	Use Case
`focus`	Element receives focus	Form field highlighting
`blur`	Element loses focus	Validation triggers
`keydown`	Key is pressed	Keyboard shortcuts
`scroll`	Element is scrolled	Infinite scroll, sticky headers

---

4. HostBinding - Binding to Host Properties

4.1 Introduction to HostBinding

The `@HostBinding` decorator binds a directive class property to a property of the host element.

4.2 Basic HostBinding Example

```
import { Directive, HostBinding, HostListener } from '@angular/core';

@Directive({
  selector: '[appDynamicBorder]'
})
export class DynamicBorderDirective {

  @HostBinding('style.border') border: string = '2px solid transparent';
  @HostBinding('style.padding') padding: string = '10px';

  @HostListener('mouseenter')
  onMouseEnter() {
    this.border = '2px solid #4CAF50';
  }

  @HostListener('mouseleave')
  onMouseLeave() {
    this.border = '2px solid transparent';
  }
}
```

Usage

```
<p appDynamicBorder>Hover to see border</p>
```

4.3 HostBinding for Classes

```
@Directive({
  selector: '[appActiveToggle]'
})
export class ActiveToggleDirective {

  private isActive = false;

  @HostBinding('class.active') get active() {
    return this.isActive;
  }

  @HostListener('click')
  toggleActive() {
    this.isActive = !this.isActive;
  }
}
```

4.4 Combining HostListener and HostBinding

```
@Directive({
  selector: '[appButtonState]'
})
export class ButtonStateDirective {

  @HostBinding('disabled') isDisabled = false;
  @HostBinding('class.loading') isLoading = false;
  @HostBinding('style.cursor') cursor = 'pointer';

  @HostListener('click')
  onClick() {
    // Simulate async operation
    this.isLoading = true;
    this.isDisabled = true;
    this.cursor = 'wait';
```

```
    setTimeout(() => {
      this.isLoading = false;
      this.isDisabled = false;
      this.cursor = 'pointer';
    }, 2000);
  }
}
```

4.5 Modern Alternative: host Metadata (Angular 15+)

Angular 15+ introduced the `host` property in the directive decorator as a cleaner alternative to `@HostListener` and `@HostBinding` decorators.

> **Modern Approach:** The `host` metadata provides a more declarative way to define host bindings and listeners without decorators.

Traditional vs Modern Approach

Traditional with Decorators

```
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @HostBinding('style.backgroundColor') bgColor = 'transparent';
  @HostBinding('class.highlighted') isHighlighted = false;

  @HostListener('mouseenter')
  onMouseEnter() {
    this.bgColor = 'yellow';
    this.isHighlighted = true;
  }

  @HostListener('mouseleave')
  onMouseLeave() {
    this.bgColor = 'transparent';
    this.isHighlighted = false;
  }

  @HostListener('click', ['$event'])
  onClick(event: MouseEvent) {
```

```
    console.log('Clicked at:', event.clientX, event.clientY);
  }
}
```

Modern with host Metadata

```
import { Directive } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true,
  host: {
    // Property bindings
    '[style.backgroundColor]': 'bgColor',
    '[class.highlighted]': 'isHighlighted',

    // Event listeners
    '(mouseenter)': 'onMouseEnter()',
    '(mouseleave)': 'onMouseLeave()',
    '(click)': 'onClick($event)',

    // Static attributes
    'role': 'button',
    'tabindex': '0'
  }
})
export class HighlightDirective {
  bgColor = 'transparent';
  isHighlighted = false;

  onMouseEnter() {
    this.bgColor = 'yellow';
    this.isHighlighted = true;
  }

  onMouseLeave() {
    this.bgColor = 'transparent';
    this.isHighlighted = false;
  }

  onClick(event: MouseEvent) {
```

```
    console.log('Clicked at:', event.clientX, event.clientY);
  }
}
```

Comprehensive host Metadata Example

```
import { Directive, Input } from '@angular/core';

@Directive({
  selector: '[appButton]',
  standalone: true,
  host: {
    // Class bindings
    '[class.btn]': 'true',
    '[class.btn-primary]': 'variant === "primary"',
    '[class.btn-secondary]': 'variant === "secondary"',
    '[class.btn-disabled]': 'disabled',
    '[class.btn-loading]': 'loading',

    // Style bindings
    '[style.padding]': 'size === "large" ? "12px 24px" : "8px 16px"',
    '[style.fontSize]': 'size === "large" ? "16px" : "14px"',
    '[style.cursor]': 'disabled ? "not-allowed" : "pointer"',
    '[style.opacity]': 'disabled ? "0.6" : "1"',

    // Attribute bindings
    '[attr.disabled]': 'disabled || null',
    '[attr.aria-disabled]': 'disabled',
    '[attr.aria-busy]': 'loading',

    // Event listeners
    '(click)': 'handleClick($event)',
    '(mouseenter)': 'onHover(true)',
    '(mouseleave)': 'onHover(false)',
    '(focus)': 'onFocus()',
    '(blur)': 'onBlur()'
  }
})
export class ButtonDirective {
  @Input() variant: 'primary' | 'secondary' = 'primary';
  @Input() size: 'normal' | 'large' = 'normal';
  @Input() disabled = false;
  @Input() loading = false;

  handleClick(event: MouseEvent) {
```

```
    if (this.disabled || this.loading) {
      event.preventDefault();
      event.stopPropagation();
      return;
    }
    console.log('Button clicked');
  }

  onHover(isHovering: boolean) {
    console.log('Hover state:', isHovering);
  }

  onFocus() {
    console.log('Button focused');
  }

  onBlur() {
    console.log('Button blurred');
  }
}
```

Usage in Template

```
<button appButton variant="primary" size="large">Primary Button</button>
<button appButton variant="secondary" [disabled]="true">Disabled Button</button>
<button appButton [loading]="isLoading">Loading Button</button>
```

Output:

Browser Display:

Three styled buttons appear with appropriate classes and styles applied:

1. Large primary button with 12px padding and 16px font
2. Secondary button with disabled styling (opacity 0.6, cursor not-allowed)
3. Button showing loading state with aria-busy attribute

Hover, click, focus, and blur events are logged to console.

Benefits of host Metadata

Feature	Decorator Approach	host Metadata
Syntax	Multiple decorators scattered in class	All bindings in one place
Readability	Harder to see all host interactions	Centralized, easier to review
Static attributes	Requires @HostBinding with string value	Direct key-value pairs
Template syntax	N/A	Uses familiar template syntax in metadata
Performance	Same	Same (compiled identically)

---

5. Renderer2 - Safe DOM Manipulation

5.1 Why Use Renderer2?

Renderer2 provides a safe way to manipulate the DOM that works across different platforms (browser, server-side rendering, web workers).

> **Best Practice:** Always use Renderer2 instead of directly accessing `nativeElement` for DOM manipulation. Direct DOM access can cause security issues and breaks server-side rendering.

5.2 Basic Renderer2 Usage

```
import { Directive, ElementRef, Renderer2, OnInit } from '@angular/core';

@Directive({
  selector: '[appStyledBox]'
})
export class StyledBoxDirective implements OnInit {

  constructor(
```

```
    private el: ElementRef,
    private renderer: Renderer2
  ) {}

  ngOnInit() {
    // Set styles
    this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', '#e3f2fd');
    this.renderer.setStyle(this.el.nativeElement, 'padding', '20px');
    this.renderer.setStyle(this.el.nativeElement, 'borderRadius', '8px');

    // Add class
    this.renderer.addClass(this.el.nativeElement, 'styled-box');

    // Set attribute
    this.renderer.setAttribute(this.el.nativeElement, 'data-styled', 'true');
  }
}
```

5.3 Renderer2 Methods

Method	Purpose	Example
`setStyle()`	Set inline style	`renderer.setStyle(el, 'color', 'red')`
`removeStyle()`	Remove inline style	`renderer.removeStyle(el, 'color')`
`addClass()`	Add CSS class	`renderer.addClass(el, 'active')`
`removeClass()`	Remove CSS class	`renderer.removeClass(el, 'active')`
`setAttribute()`	Set attribute	`renderer.setAttribute(el, 'disabled', 'true')`
`removeAttribute()`	Remove attribute	`renderer.removeAttribute(el, 'disabled')`
`setProperty()`	Set property	

Method	Purpose	Example
		`renderer.setProperty(el, 'value', 'text')`
`listen()`	Add event listener	`renderer.listen(el, 'click', fn)`

5.4 Advanced Renderer2 Example

```
@Directive({
  selector: '[appTooltip]'
})
export class TooltipDirective implements OnInit, OnDestroy {

  @Input('appTooltip') tooltipText = '';

  private tooltipElement: any;
  private mouseEnterListener: () => void;
  private mouseLeaveListener: () => void;

  constructor(
    private el: ElementRef,
    private renderer: Renderer2
  ) {}

  ngOnInit() {
    // Create event listeners
    this.mouseEnterListener = this.renderer.listen(
      this.el.nativeElement,
      'mouseenter',
      () => this.showTooltip()
    );

    this.mouseLeaveListener = this.renderer.listen(
      this.el.nativeElement,
      'mouseleave',
      () => this.hideTooltip()
    );
  }

  showTooltip() {
    // Create tooltip element
    this.tooltipElement = this.renderer.createElement('span');
```

```
    // Set text
    const text = this.renderer.createText(this.tooltipText);
    this.renderer.appendChild(this.tooltipElement, text);

    // Style tooltip
    this.renderer.setStyle(this.tooltipElement, 'position', 'absolute');
    this.renderer.setStyle(this.tooltipElement, 'backgroundColor', '#333');
    this.renderer.setStyle(this.tooltipElement, 'color', 'white');
    this.renderer.setStyle(this.tooltipElement, 'padding', '5px 10px');
    this.renderer.setStyle(this.tooltipElement, 'borderRadius', '4px');
    this.renderer.setStyle(this.tooltipElement, 'fontSize', '12px');
    this.renderer.setStyle(this.tooltipElement, 'zIndex', '1000');

    // Add to DOM
    this.renderer.appendChild(document.body, this.tooltipElement);

    // Position tooltip
    const hostPos = this.el.nativeElement.getBoundingClientRect();
    const tooltipPos = this.tooltipElement.getBoundingClientRect();

    const top = hostPos.top - tooltipPos.height - 10;
    const left = hostPos.left + (hostPos.width - tooltipPos.width) / 2;

    this.renderer.setStyle(this.tooltipElement, 'top', `${top}px`);
    this.renderer.setStyle(this.tooltipElement, 'left', `${left}px`);
  }

  hideTooltip() {
    if (this.tooltipElement) {
      this.renderer.removeChild(document.body, this.tooltipElement);
      this.tooltipElement = null;
    }
  }

  ngOnDestroy() {
    // Clean up listeners
    if (this.mouseEnterListener) {
      this.mouseEnterListener();
    }
    if (this.mouseLeaveListener) {
      this.mouseLeaveListener();
    }
    this.hideTooltip();
  }
}
```

Usage

```
<button appTooltip="Click to save changes">Save</button>
<p appTooltip="This is helpful information">Hover for tooltip</p>
```

Output:

Browser Display:

When hovering over elements with the tooltip directive, a dark tooltip appears above the element showing the specified text. The tooltip is positioned dynamically and disappears when the mouse leaves.

---

6. Modern Directive Features (Angular 14+)

6.1 Standalone Directives

Angular 14+ introduced standalone directives, which don't require NgModule declarations and can be imported directly where needed.

> **Modern Pattern:** Standalone directives simplify the module system and make directives more portable and reusable.

Creating a Standalone Directive

```
// highlight.directive.ts
import { Directive, ElementRef, Input, OnInit } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true  // 👈 Makes it standalone
})
export class HighlightDirective implements OnInit {
  @Input() appHighlight = 'yellow';
  @Input() defaultColor = 'transparent';
```

```
  constructor(private el: ElementRef) {}

  ngOnInit() {
    this.el.nativeElement.style.backgroundColor = this.appHighlight;
  }
}
```

Using Standalone Directives

In a Standalone Component

```
// app.component.ts
import { Component } from '@angular/core';
import { HighlightDirective } from './highlight.directive';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [HighlightDirective],  // 👍 Import directly
  template: `
    <p appHighlight="lightblue">This text is highlighted</p>
    <div [appHighlight]="color">Dynamic highlight</div>
  `
})
export class AppComponent {
  color = 'lightgreen';
}
```

In a Module-Based Component

```
// app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { HighlightDirective } from './highlight.directive';

@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    HighlightDirective  // 👍 Can import standalone directive in module
```

```
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Complete Standalone Directive Example

```
// clickable.directive.ts
import { Directive, Output, EventEmitter } from '@angular/core';

@Directive({
  selector: '[appClickable]',
  standalone: true,
  host: {
    '[class.clickable]': 'true',
    '[style.cursor]': '"pointer"',
    '[style.userSelect]': '"none"',
    '[tabindex]': '0',
    '(click)': 'onClick($event)',
    '(keydown.enter)': 'onEnter($event)',
    '(keydown.space)': 'onSpace($event)'
  }
})
export class ClickableDirective {
  @Output() appClickable = new EventEmitter<MouseEvent | KeyboardEvent>();

  onClick(event: MouseEvent) {
    this.appClickable.emit(event);
  }

  onEnter(event: KeyboardEvent) {
    event.preventDefault();
    this.appClickable.emit(event);
  }

  onSpace(event: KeyboardEvent) {
    event.preventDefault();
    this.appClickable.emit(event);
  }
}
```

```
// app.component.ts
import { Component } from '@angular/core';
import { ClickableDirective } from './clickable.directive';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [ClickableDirective],
  template: `
    <div appClickable (appClickable)="handleClick($event)">
      Click me or press Enter/Space
    </div>
  `,
  styles: [`
    .clickable {
      padding: 20px;
      background: #e3f2fd;
      border-radius: 8px;
      display: inline-block;
    }
    .clickable:hover {
      background: #bbdefb;
    }
  `]
})
export class AppComponent {
  handleClick(event: MouseEvent | KeyboardEvent) {
    console.log('Clicked!', event.type);
  }
}
```

6.2 Modern Class and Style Binding

Angular provides multiple ways to bind classes and styles with improved syntax and type safety.

Enhanced Class Binding

```
<!-- Single class toggle -->
<div [class.active]="isActive">Single class</div>

<!-- Multiple classes with ngClass -->
<div [ngClass]="{ active: isActive, disabled: isDisabled }">Object syntax</div>
```

```
<!-- String interpolation (avoid if possible) -->
<div class="base {{ dynamicClass }}">String interpolation</div>

<!-- Modern: Class binding with array -->
<div [class]="['base-class', isActive ? 'active' : '', extraClass]">
  Array syntax (Angular 9+)
</div>

<!-- Modern: Direct class binding -->
<div [class]="getClasses()">Method returns class string/array/object</div>
```

Enhanced Style Binding

```
<!-- Single style -->
<div [style.color]="textColor">Colored text</div>

<!-- Style with unit -->
<div [style.width.px]="widthValue">Width in pixels</div>
<div [style.height.%]="heightPercent">Height in percent</div>

<!-- Multiple styles with ngStyle -->
<div [ngStyle]="{ color: textColor, 'font-size.px': fontSize }">
  Object syntax
</div>

<!-- Modern: Direct style binding with object -->
<div [style]="{ color: textColor, fontSize: fontSize + 'px' }">
  Direct style object (Angular 9+)
</div>

<!-- Modern: Style binding with method -->
<div [style]="getStyles()">Method returns style object</div>
```

Component Example

```
import { Component } from '@angular/core';
import { NgClass, NgStyle } from '@angular/common';

@Component({
  selector: 'app-styled',
  standalone: true,
  imports: [NgClass, NgStyle],
```

```
  template: `
    <h3>Modern Binding Syntax</h3>

    <!-- Single bindings -->
    <div [class.highlighted]="isHighlighted"
         [style.backgroundColor]="bgColor"
         [style.padding.px]="20">
      Single bindings
    </div>

    <!-- Multiple bindings -->
    <div [class]="cardClasses"
         [style]="cardStyles">
      Direct object bindings
    </div>

    <!-- Traditional directives -->
    <div [ngClass]="getClasses()"
         [ngStyle]="getStyles()">
      Traditional directive syntax
    </div>

    <button (click)="toggle()">Toggle State</button>
  `
})
export class StyledComponent {
  isHighlighted = true;
  bgColor = '#e3f2fd';

  cardClasses = {
    'card': true,
    'card-elevated': true,
    'card-primary': this.isHighlighted
  };

  cardStyles = {
    padding: '20px',
    borderRadius: '8px',
    boxShadow: '0 2px 4px rgba(0,0,0,0.1)'
  };

  toggle() {
    this.isHighlighted = !this.isHighlighted;
    this.bgColor = this.isHighlighted ? '#e3f2fd' : '#f5f5f5';
  }

  getClasses() {
```

```
    return {
      active: this.isHighlighted,
      inactive: !this.isHighlighted
    };
  }

  getStyles() {
    return {
      'font-weight': this.isHighlighted ? 'bold' : 'normal',
      'color': this.isHighlighted ? '#1976d2' : '#666'
    };
  }
}
```

6.3 Type-Safe Directive Inputs (Angular 16+)

Angular 16+ introduced required inputs and transform functions for better type safety.

```
import { Directive, Input, booleanAttribute, numberAttribute } from '@angular/core';

@Directive({
  selector: '[appValidator]',
  standalone: true
})
export class ValidatorDirective {
  // Required input - must be provided
  @Input({ required: true }) fieldName!: string;

  // Boolean attribute with transform
  @Input({ transform: booleanAttribute }) required = false;

  // Number attribute with transform
  @Input({ transform: numberAttribute }) maxLength = 100;

  // Custom transform
  @Input({
    transform: (value: string | string[]) =>
      Array.isArray(value) ? value : value.split(',')
  })
  allowedValues: string[] = [];
}
```

Usage

```
<!-- fieldName is required, will error if missing -->
<input appValidator fieldName="email">

<!-- Boolean attributes work without value -->
<input appValidator fieldName="username" required>

<!-- Numbers are automatically converted -->
<input appValidator fieldName="password" maxLength="50">

<!-- String is automatically split into array -->
<input appValidator
       fieldName="country"
       allowedValues="US,UK,CA">
```

Output:

Benefits:

- **Type Safety:** Transforms ensure correct types at runtime
- **Required Inputs:** Compile-time errors for missing required inputs
- **Cleaner Templates:** Boolean attributes don't need `="true"`
- **Automatic Conversion:** String attributes converted to appropriate types

---

7. Best Practices

6.1 Choosing the Right Tool

Tool	Use When	Avoid When
ngClass	Multiple CSS classes need conditional application	Only one class needs to be toggled (use `[class.name]`)
ngStyle		

Tool	Use When	Avoid When
	Multiple inline styles with dynamic values	Static styles (use CSS) or single style (use `[style.prop]`)
HostListener	Need to respond to host element events in directive	Component template events (use template event binding)
HostBinding	Binding directive property to host element property	Template bindings (use template syntax)
Renderer2	Any DOM manipulation in directives	Never avoid—always prefer over direct DOM access

7.3 Performance Tips

- Prefer CSS classes over inline styles when possible
- Use `ngClass` and `ngStyle` instead of multiple individual bindings
- Clean up event listeners in `ngOnDestroy`
- Avoid heavy computations in style/class getter methods
- Use Renderer2 for all DOM manipulations
- Prefer standalone directives for new code (Angular 14+)
- Use host metadata instead of decorators for cleaner code
- Leverage transform functions for input type safety

7.4 Common Mistakes

1. **Direct DOM manipulation**

```
// ❌ Wrong - Security risk, breaks SSR
this.el.nativeElement.style.color = 'red';

// ☑ Correct - Use Renderer2
this.renderer.setStyle(this.el.nativeElement, 'color', 'red');
```

2. **Not cleaning up listeners**

```
// ❌ Wrong - Memory leak
@HostListener('click')
onClick() { }

// ☑ Correct - Clean up manual listeners
ngOnDestroy() {
  if (this.listener) {
    this.listener(); // Unlisten
  }
}
```

3. **Complex expressions in ngClass/ngStyle**

```
<!-- ❌ Wrong - Hard to read -->
[ngClass]="user.role === 'admin' && user.active && !user.suspended ? 'admin-active' :
'user-default'"

<!-- ☑ Correct - Use component method -->
[ngClass]="getUserClasses()"
```

Summary

Attribute directives enhance element appearance and behavior without changing DOM structure:

Traditional Approach

1. **ngClass:** Dynamically apply CSS classes using objects, arrays, or strings
2. **ngStyle:** Set multiple inline styles dynamically based on component data
3. **@HostListener:** Decorator to listen to host element events (click, hover, focus, etc.)
4. **@HostBinding:** Decorator to bind directive properties to host element properties
5. **Renderer2:** Service for safe, platform-agnostic DOM manipulation

Modern Approach (Angular 14+)

1. **Standalone Directives:** Self-contained directives without NgModule declarations
2. **host Metadata:** Cleaner alternative to @HostListener and @HostBinding decorators
3. **Direct Bindings:** [class] and [style] bindings for simpler cases instead of ngClass/ngStyle
4. **Type-Safe Inputs:** Transform functions and required inputs for better type safety
5. **Centralized Configuration:** All host interactions defined in directive metadata

For new Angular projects, prefer the modern approach with standalone directives and host metadata. These tools form the foundation for creating custom attribute directives, which we'll explore in the next note.

End of Angular Note 4.3: Attribute Directives